

**Universität Leipzig
Institut für Informatik**

**Vergleich zweier Varianten der Lösungserzeugung von
Ameisenalgorithmen zur Funktionsoptimierung**

Bachelorarbeit

Leipzig, Oktober, 2019

vorgelegt von

Felix Cäsar Madorskiy
Studiengang Informatik B.Sc.

**Betreuender Hochschullehrer: Prof. Dr. Martin Middendorf
Fakultät für Mathematik und Informatik/Institut für Informatik/
Schwarmintelligenz und Komplexe Systeme**

Inhaltsverzeichnis

1. Einleitung	1
2. Einführung in Ameisenalgorithmen	2
2.1. Beschreibung der Idee eines Ameisenalgorithmen	2
2.2. mathematische Beschreibung	3
2.3. Permutationsprobleme	4
2.4. Verbesserungen der ACO Metaheuristik zur PB-ACO	4
2.5. Ameisenalgorithmen mit stetigem Lösungsarchiv	5
2.6. Ameisenalgorithmen in stetigen Variablen	5
3. Varianten der Lösungsfindung	6
3.1. $ACO_{\mathbb{R}}$	6
3.2. $ACO_{\mathbb{R}}$ -simple	8
3.3. $ACO_{\mathbb{R}}$ -very-simple	9
4. Testfunktion	10
5. Einfluss der Parameter	15
5.1. Einfluss der Anzahl der Ameisen	16
5.2. Einfluss des Parameters q	16
5.3. Einfluss des Parameters k	17
5.4. Einfluss des Parameters ξ	18
5.5. Auswertung der Einflüsse der Parameter	18
6. Vergleich zweier Intervallberechnungen für $ACO_{\mathbb{R}}$-simple und $ACO_{\mathbb{R}}$-very-simple	19
7. Vergleich von $ACO_{\mathbb{R}}$, $ACO_{\mathbb{R}}$-simple und $ACO_{\mathbb{R}}$-very-simple	21
8. Konvergenzbetrachtung	28
8.1. Rosenbrock	29
9. Kurzzusammenfassung	30
Literaturverzeichnis	31

Anlagen

- A. Einfluss der Anzahl der Ameisen
- B. Einfluss des Parameters q
- C. Einfluss des Parameters ξ

1. Einleitung

Die Informatik als Wissenschaft gliedert sich in viele Bereiche, eines davon ist das Entwickeln von Algorithmen zum heuristischen Berechnen von Näherungslösungen von Problemen, von denen ausgegangen wird, dass es für sie keine effizienten Algorithmen gibt und somit eine Näherungslösung das Beste ist, was erreicht werden kann als Kompromiss. Ferner gibt es keine Möglichkeit zur Verifizierung, ob diese so gefundenen Lösungen tatsächlich Lösungen zu den Problemen sind, sie sind nur gut genug um in der Praxis stellvertretend als angenäherte Lösungen zu dienen.

Die, für das heuristische Berechnen von Lösungen, verwendeten Algorithmen können weiter aufgeteilt werden in Klassen von Algorithmen. Die hier relevante Klasse sind Ameisenalgorithmen, um genau zu sein, Ameisenalgorithmen in stetigen Variablen. Diese Klassen können weiter aufgeteilt werden in Varianten zur Lösungserzeugung. Konkret beschäftigt sich diese Arbeit mit dem Vergleich zweier Varianten der Lösungserzeugung von Ameisenalgorithmen, die erste Variante repräsentiert durch den Algorithmus $\text{ACO}_{\mathbb{R}}$ und die andere durch die Algorithmen $\text{ACO}_{\mathbb{R}}\text{-simple}$ und $\text{ACO}_{\mathbb{R}}\text{-very-simple}$.

Ameisenalgorithmen begannen ihre Entwicklung durch die Inspiration von dem Verhalten von Ameisen bei der Nahrungssuche. Dadurch waren die ersten Ameisenalgorithmen dafür gedacht, Anordnungsprobleme, Permutationsprobleme, zu lösen, worin sie sehr erfolgreich waren, z.B. bei der Routenplanung für Fahrzeuge. Sie hatten nur ein Nachteil, sie waren nur für diskrete Variablen einsetzbar, also Variablen, die nur endlich viele Werte annehmen konnten. Dies limitierte jedoch den Einsatzbereich der Algorithmen für Probleme, wie sie in der Praxis oft vorkommen, Probleme in stetigen Variablen, also Probleme mit Variablen, die beliebige Werte annehmen können.

Dieses Hindernis wurde jedoch bereits überwunden, durch gleich mehrere Varianten der Lösungserzeugung von Ameisenalgorithmen, u.a. Continuous ACO (CACO) vorgestellt in *Bilchev and Parmee* [1] und dem hier betrachteten Algorithmus $\text{ACO}_{\mathbb{R}}$ vorgestellt in *Socha und Dorigo* [6]. Doch alle Varianten der Lösungserzeugung in stetigen Variablen, welche durch Ameisenalgorithmen inspiriert wurden, zeichnen sich dadurch aus, dass ihre Verwendung von bereits gefundenen Lösungen und das Zusammensetzen dieser zu neuen Lösungen eine gewisse Komplexität aufweist.

Genau mit diesem Problem beschäftigt sich die hier vorliegende Arbeit durch das Vorstellen einer sehr stark vereinfachten Variante zur Lösungserzeugung eines Ameisenalgorithmus und dem Vergleich dieser Variante mit der, wie das Paper von *Socha und Dorigo* [6] selbst aussagt, ACO im Geiste treuesten Variante, $\text{ACO}_{\mathbb{R}}$. Wir beantworten also in dieser Arbeit die Frage, durch den Vergleich der beiden Varianten, ob die Komplexität wirklich nötig ist.

2. Einführung in Ameisenalgorithmen

2.1. Beschreibung der Idee eines Ameisenalgorithmen

Ameisenalgorithmen sind durch die Art und Weise des Verhaltens, z.B. Nahrungssuche, von Ameisen inspiriert. Dabei hinterlassen Ameisen, bei dem Zurücklegen der Strecke zwischen zwei Punkten A und B, eine Pheromonspur, an der sich Artgenossen orientieren können. Ameisen entscheiden sich jedoch nur mit einer gewissen Wahrscheinlichkeit für den Weg mit der Pheromonspur.

Auf der anderen Seite verdunstet diese Spur nach einer gewissen Zeit, sodass einmal gelegte Wege auch wieder verschwinden können.

Nur wenn Wege zu einem Ziel mit hoher Priorität führen, wandern viele Ameisen auf ihnen und die Pheromonspur wird hinreichend oft erneuert, sodass mehr Pheromon gelegt wird als verdunstet, wodurch eine Straße entsteht, auf der eine große Anzahl Ameisen laufen.

Findet eine Ameise einen kürzeren Weg, so legt sie ebenfalls auch auf diesem Pheromon ab und sobald sich noch weitere Ameisen für den kürzeren Weg entscheiden, wird dieser von andere Ameisen verstärkt bevorzugt, da, je kürzer der Weg, desto schneller können die Ameisen wieder zurückkehren und damit wiederum schneller beim Zurückkehren weiteres Pheromon auf dem Weg ablegen, sodass wiederum andere Ameisen sich eher für den Weg entscheiden, da auf dem kürzeren Weg jetzt mehr Pheromon liegt.

Dies ist zunächst so umgesetzt worden, dass, wenn ein Suchagent, im Folgenden, um bei der Analogie der Ameisen zu bleiben, Ameise genannt, die Wahl zwischen mehreren Wegen hat, er sich eher für einen Weg entscheidet, auf dem bereits eine Pheromonspur bzw. mehr Pheromon liegt, jedoch nur mit einer gewissen Wahrscheinlichkeit. Nach jedem Schritt/Zeiteinheit wird zudem Pheromon verdunstet.

2.2. mathematische Beschreibung

Die folgende Beschreibung ist nach *Prof. Dr. M. Middendorf* [4].

Seien also drei Punkte gegeben, zwischen denen Ameisen sich bewegen können – ein Startpunkt (S), ein Zielpunkt (Z) und ein Punkt (U) als Umweg. Die Ameisen können hin und her auf der Verbindung Start–Ziel direkt laufen oder zuerst über den dritten Punkt U und erst dann in Z ankommen.

Die Wahrscheinlichkeit sich für den Umweg zu entscheiden beschreibt Gleichung (1), die, für den direkten Weg SZ, Gleichung (2). Dabei ist α ein Parameter, der den Einfluss von Pheromon auf einem Pfad steuert.

Der direkt Weg bedarf eines Schritts/Zeiteinheit, das Laufen über den Umweg, zwei. Bei jedem Schritt wird Pheromon gemäß Gleichung (3) verdunstet, wobei v die Verdunstungskonstante ist.

$$prob_{SU} = \frac{\rho_{SU}^\alpha}{\rho_{SU}^\alpha + \rho_{SZ}^\alpha} \quad (1)$$

$$prob_{SZ} = \frac{\rho_{SZ}^\alpha}{\rho_{SU}^\alpha + \rho_{SZ}^\alpha} \quad (2)$$

$$\rho_{XY} = (1 - v) \cdot \rho_{XY}, \quad v \in [0, 1), \quad XY \in \{SU, UZ, SZ\} \quad (3)$$

Die Pheromoninformation wird hier, von einem Konstruktionsdurchgang der Lösung zum nächsten, weiter gereicht.

Wie zu sehen ist, konstruiert der beschriebene Algorithmus eine Anordnung der Punkte, also von welchem Punkt zu welchem gegangen werden soll und in welcher Reihenfolge, um die kürzeste Verbindung zwischen Startpunkt S und Zielpunkt Z zu finden – hier die direkte Verbindung SZ. Wir bemerken weiter, dass nur eine einzige Belegung der Punkte mögliche ist und die Belegung ist auf die Punkte beschränkt. Entweder wird ein Punkt begangen oder nicht, so wie U. Und es gibt auch keine andere Möglichkeit, außer SZ, zwischen S und Z zu gehen, es sei denn es ist über U.

Ein Problem bei dem nur eine abzählbare Belegung der Punkte oder, verallgemeinert ausgedrückt, Variablen mögliche ist, heißt diskretes Problem. Dabei bedeutet abzählbar, dass die Menge der Werte, welche die Variablen annehmen können, wie die natürlichen Zahlen, abzählbar ist bzw. die Belegungen der Variablen gezählt werden können – hier gibt es nur zwei, begangen und nicht begangen.

Zusammenfassend lässt sich also sagen, die ersten Ameisenalgorithmen waren Algorithmen, Metaheuristiken, zum Lösen diskreter Probleme, also Probleme deren Variablen nur endlich viele Werte annehmen konnten. Um genau zu sein waren es Anordnungs-

bzw. Permutationsprobleme. Diese lassen sich wie folgt definieren.

2.3. Permutationsprobleme

Ein Permutationsproblem ist gegeben durch das Tupel (D, S, B, f) . Dabei ist $D = \{D_1, D_2, \dots, D_n\}, n \in \mathbb{N}$, die Menge an Werten, welche die diskreten Variablen $X_1, X_2, \dots, X_n, n \in \mathbb{N}$, aus den endlichen Mengen D_i annehmen können, S der Suchraum, $B = \{B_1, B_2, \dots, B_n\}, n \in \mathbb{N}$ die Menge der Beschränkungen, welche an die Variablenbelegungen gestellt werden und $f : S \rightarrow \mathbb{R}_0^+$ ist die zu minimierende Ziel- bzw. Kostenfunktion. Wenn B leer ist, heißt das Problem unbeschränkt, sonst beschränkt.

Eine Lösung $s \in S$ ist das Belegen aller Variablen X_i mit Werten aus D_i , welche konform mit den Beschränkungen aus B sind.

Eine Lösung s_{opt} ist ein Optimum, wenn $f(s_{opt}) \leq f(s) \forall s \in S$.

2.4. Verbesserungen der ACO Metaheuristik zur PB-ACO

Die folgende Beschreibung ist nach *Prof. Dr. M. Middendorf* [4].

Da festgestellt wurde, dass bei Iterationen der ACO Metaheuristik viel Zeit ($O(n^2)$) für das Update der Pheromoninformation aufgewendet wird, wurde von *Gutsch und Middendorf* [2] ein effizienteres Verfahren vorgestellt, bei dem das Pheromonupdate nur $O(n)$ benötigt. Dabei wird Pheromon in einer diskreten, quadratischen $(n \times n)$ -Matrix P , $n \in \mathbb{N}$, gespeichert, mit Einträgen, die sich wie folgt zusammensetzen:

Seien $\rho_0, \delta > 0$ und $P \in \mathbb{R}^{n \times n}, n \in \mathbb{N}$. Dann gilt:

$$\rho_{ij} = \rho_0 \quad \forall i, j \in \{1, \dots, n\} \quad (4)$$

Alle Einträge der Pheromonmatrix P werden also zu Beginn auf einen Initialwert ρ_0 gesetzt. Danach konstruieren die Ameisen mehrere Lösungen und die best wird in die Pheromonmatrix eingefügt gemäß Gleichung (5).

$$\rho_{ij} = \rho_{ij} + \delta, \quad (5)$$

wenn die (i, j) -Kante eine Kante in der Lösung ist.

Dadurch gilt für alle Einträge der Pheromonmatrix:

$$\rho_0 \leq \rho_{ij} \leq \rho_0 + n \cdot \delta \quad (6)$$

und jeder Matrix- bzw. Pheromoneintrag kann genau $n + 1$ Werte annehmen. Deswegen ist P auch eine diskrete Matrix, da ihre Einträge nur endliche viele Werte annehmen

können.

Zudem gilt, dass in jedem Durchlauf, abgesehen von den ersten n , eine Lösung in die diskrete Pheromonmatrix aufgenommen und eine aus selbiger wieder entfernt wird. Es gilt also, wenn man die Matrix P als solches interpretiert, dass P ein Lösungsarchiv ist, mit dessen Hilfe eine neue Lösung erzeugt wird.

Diese Verbesserung ist auch ein wesentlicher Schritt in Richtung der Algorithmen, welche dieser Arbeit beiliegen und auch des Algorithmus mit dem der Vergleich in dieser Arbeit gezogen wird.

2.5. Ameisenalgorithmen mit stetigem Lösungsarchiv

Wie in dem vorherigen Abschnitt beschrieben, kann man, statt die Pheromoninformation von Iterationsschritt zu Iterationsschritt mitzuführen, die Information auch in einer diskreten Matrix speichern, also einem diskreten Lösungsarchiv. Dies funktioniert jedoch nur für diskrete Probleme. Um stetige Probleme zu lösen, kann man auf ein Archiv mit stetigen Einträgen zurückgreifen, was dann eine u.a. eine Liste sein kann.

Ameisenalgorithmen mit einem stetigen Lösungsarchiv speichern ihre Lösungen in stetigen Einträgen. Wie genau, hängt von dem jeweiligen Algorithmus ab. Die einfachste Art und Weise dies zu tun, ist jedoch, die Lösungen wie sie generiert werden, nach einer bestimmten Vorschrift zu speichern, also ein einfaches Lösungsarchiv zu verwenden.

Wiederum die einfachsten Vorschriften zum Speichern von Lösungen sind „first in last out“ und „best in worst out“. Bei der ersten Vorschrift wird die neueste Lösung ins Archiv aufgenommen und die Älteste entfernt und bei der Zweiten wird die Lösung nach ihrer Güte einsortiert. Ist bei der zweiten Vorschrift die Lösung schlechter als alle anderen im Archiv, wird die Lösung verworfen, ist sie besser als eine gespeicherte Lösung, wird die neue gute Lösung an die Stelle der alten schlechteren Lösung gesetzt, alle Lösungen ab der schlechteren Lösung, die schlechtere Lösung eingeschlossen, werden um einen Platz nach hinten verschoben und die schlechteste Lösung wird aus dem Archiv gelöscht.

Bei den dieser Arbeit beiliegenden Algorithmen wird ein einfaches Lösungsarchiv verwendet und nach dem „best in worst out“ Prinzip befüllt.

2.6. Ameisenalgorithmen in stetigen Variablen

Ameisenalgorithmen in stetigen Variablen suchen Belegungen für jede ihrer Variablen in $\mathbb{R}$ oder in einem Intervall von $\mathbb{R}$, der Suchraum ist also z.B. $\mathbb{R}^n, n \in \mathbb{N}$.

Da für ein stetiges Problem keine Matrix mit diskreten Einträgen verwendet werden kann, ist es möglich, wie in *Socha und Dorigo* [6] und dem vorherigen Abschnitt be-

schrieben, ein Lösungsarchiv zu verwendet, in dem die Lösungen gespeichert werden.

Aus den Lösungen im Archiv wird dann eine neue Lösung konstruiert. Dies kann auf unterschiedlichste Weise passieren, wie in den Abschnitten 3.1, 3.2 und 3.3 beschrieben.

Das Beschreiben andere Methoden der Generierung und Verarbeitung von Lösungen, abgesehen vom Speichern in einem Lösungsarchiv und der Konstruktion von neuen Lösungen aus diesem, geht über den Umfang dieser Arbeit hinaus und wird nicht betrachtet.

3. Varianten der Lösungsfindung

Das dieser Arbeit beiliegende Programm ist ein Intervallauswertungsalgorithmus. Wie in der Einleitung motiviert, fragen wir uns, ob komplexe Lösungsfindungsmethoden wirklich nötig sind, um gute Lösungen zu finden, unabhängig davon, ob wir bereits vorhandene Lösungen zu Neuen zusammensetzen oder komplett neue Lösungen auf eine andere Art und Weise generieren.

Deswegen, anstatt wie *Socha und Dorigo* [6], wo eine Funktion über einem Intervall aufgebaut und dann ausgewertet wird, nehmen wir einfach das Intervall und werten in gewisser Weise nur das Intervall selbst aus. Dabei nehmen wir einen zufällig bestimmten Punkt aus dem Intervall und nennen dies Auswerten. Daher auch Intervallauswertungsalgorithmus.

Insbesondere sei hier festgehalten, dass dies als Variante der Lösungserzeugung interpretiert wird, also dass wir ein Intervall nehmen und dann daraus einen zufälligen Punkt und nicht die Methode mit der wir das Intervall berechnen, davon gibt es nämlich zwei. Die andere Variante ist das Aufbauen einer Funktion über einem Intervall und anschließend das Auswerten dieser Funktion, was in [6] durch $\text{ACO}_{\mathbb{R}}$ getan wird. Daher auch der Titel dieser Bachelorarbeit – es werden zwei Varianten der Lösungserzeugung verglichen, nicht Algorithmen.

Die Variante zur Lösungserzeugung, durch das Auswerten einer Funktion, wird im Abschnitt 3.1 beschrieben. Die Variante zur Lösungserzeugung durch das Auswerten eines Intervalls folgt, realisiert durch die beiden Algorithmen $\text{ACO}_{\mathbb{R}}\text{-simple}$ und $\text{ACO}_{\mathbb{R}}\text{-very-simple}$, in den Abschnitten 3.2 und 3.3.

3.1. $\text{ACO}_{\mathbb{R}}$

Wie in *Socha und Dorigo* [6] dargelegt, wird in $\text{ACO}_{\mathbb{R}}$ ein Lösungsarchiv bei der Lösungssuche mitgeführt. Das Archiv hat die Größe k , es hält also zu jedem Zeitpunkt k Lösungen.

Des Weiteren führt in $\text{ACO}_{\mathbb{R}}$ jede Ameise n Konstruktionsschritte durch, wobei n die Anzahl der Dimensionen des Problems ist. Dabei wird im i -ten Konstruktionsschritt die folgende Funktion ausgewertet.

$$G^i = \sum_{l=1}^k \omega_l g_l^i(x) = \sum_{l=1}^k \omega_l \frac{1}{\sigma_l^i \sqrt{2\pi}} e^{-\frac{(x-\mu_l^i)^2}{2\sigma_l^{i2}}}, \quad i \in \{1, \dots, n\} \quad (7)$$

Für Vektoren ω , μ und σ gilt, dass sie so viele Komponenten haben, wie es Lösungen im Lösungsarchiv gibt, also $|\omega| = |\mu| = |\sigma| = k$.

Ferner werden die Parametervektoren ω , μ und σ folgendermaßen berechnet.

$$\mu = \{\mu_1^i, \dots, \mu_k^i\} = \{s_1^i, \dots, s_k^i\}, \quad i \in \{1, \dots, n\}, \quad (8)$$

wobei n die Anzahl der Dimensionen ist, i der Konstruktionsschritt und s_j^i die i -te Variable der j -ten Lösung ist. Die Gewichte für jede Lösung ergeben sich nach Gleichung (9).

$$\omega_l = \frac{1}{qk\sqrt{2\pi}} e^{-\frac{(l-1)^2}{2q^2k^2}}, \quad l \in \{1, \dots, k\}, \quad (9)$$

Demnach wird die l -te Gaußfunktion nach der Wahrscheinlichkeitsverteilung aus Gleichung (10) gewählt.

$$p_l = \frac{\omega_l}{\sum_{r=1}^k \omega_r}, \quad l \in \{1, \dots, k\} \quad (10)$$

Und die Standardabweichung im i -ten Konstruktionsschritt wird wie folgt berechnet.

$$\sigma_l^i = \xi \sum_{e=1}^k \frac{|s_e^i - s_l^i|}{k-1}, \quad l \in \{1, \dots, k\}, \quad i \in \{1, \dots, n\}. \quad (11)$$

wobei q und ξ Parameter sind. Es mag zwar die Wurzel fehlen aber in *Socha und Dorigo* [6] wurde die Standardabweichung so berechnet bzw. der Term in der Gleichung (11) als Standardabweichung bezeichnet.

Abschließend wird in Schritt i die Gaußfunktion g_l^i ausgewertet. Dies kann u.a. durch einen Zufallszahlengenerator geschehen, der Zufallszahlen gemäß einer parametrisierten Normalverteilung generieren kann.

Abschließend gehen wir noch auf die Parameter q und ξ ein. Der Parameter q beeinflusst, wie die Lösungen im Archiv in die Suche eingebunden werden. Bei kleinem q werden besseren Lösungen stärker bevorzugt. Und der Parameter ξ beeinflusst, wie schnell der Algorithmus konvergiert. Je größer ξ , desto langsamer konvergiert der Algorithmus.

3.2. ACO_ℝ-simple

Der erste dieser Arbeit beiliegende Algorithmus ACO_ℝ-simple, nutzt das Lösungsarchiv auch wie ACO_ℝ, jedoch setzt es die vorhandenen Lösungen auf eine sehr viel einfachere Weise zu einer neuen Lösung zusammen – deswegen simple, also einfach.

Dabei wird, wie in ACO_ℝ dieselbe Gewichtung der Lösungen vorgenommen. Zuerst werden die Gewichte anhand Gleichung (12) berechnet,

$$\omega_i = \frac{1}{qk\sqrt{2\pi}} e^{-\frac{(i-1)^2}{2q^2k^2}}, \quad i \in \{1, \dots, k\}, \quad (12)$$

wobei auch hier k die Anzahl der Lösungen im Lösungsarchiv ist, also die Größe des Archivs. Danach wird die Wahrscheinlichkeitsverteilung, nach welcher eine Lösung aus dem Archiv gewählt wird, gemäß Gleichung (13) berechnet.

$$p_i = \frac{w_i}{\sum_{j=1}^n w_j}, \quad i \in \{1, \dots, k\}. \quad (13)$$

Es wird also zufällig eine Lösung s_i mit der Wahrscheinlichkeit p_i aus dem Lösungsarchiv gewählt. Danach wird die lineare Abweichung in der d-ten Dimension zu dieser Lösung mit der Gleichung (14) berechnet.

$$\delta_d = \xi \sum_{j=1}^k \frac{|s_{j_d} - s_{i_d}|}{k-1}, \quad j \in \{1, \dots, k\}, \quad d \in \{1, \dots, n\}, \quad (14)$$

Dabei ist n die Anzahl der Dimensionen des Problems.

Anschließend wird für jede Dimension ein zufälliger Punkt aus dem Intervall

$$[s_d - \delta_d, s_d + \delta_d], \quad d \in \{1, \dots, n\}, \quad (15)$$

genommen. Damit haben wir, ausgehen von einem Lösungsarchiv, eine neue Lösung gewonnen.

Die Standardeinstellungen der Parameter sind, wie in Tabelle 1 zu sehen ist.

Tabelle 1: Parametereinstellungen ACO_ℝ-simple

Parameter	Symbol	Wert
Anzahl der Ameisen	m	1
Konvergenzgeschwindigkeit	ξ	0.95
Lokalität der Suche	q	0.95
Archivgröße	k	100

3.3. ACO_ℝ-very-simple

Der zweite dieser Arbeit beiliegende Algorithmus ACO_ℝ-very-simple, nutzt ebenfalls das Lösungsarchiv wie ACO_ℝ, jedoch setzt es die vorhandenen Lösungen noch einfacher, geradezu trivial, zu einer neuen Lösung zusammen – deswegen very-simple, also sehr einfach.

Bei diesem Algorithmus wird immer nur die erste, also die beste Lösung aus dem Archiv zur Intervallberechnung benutzt, also entfällt alles, bis auf die Intervallberechnung und das Bestimmen eines zufälligen Punktes aus dem errechneten Intervall. Das heißt, die Gewichtung der Lösungen entfällt, genau wie das Berechnen der Wahrscheinlichkeitsverteilung für das Auswählen einer Lösung aus dem Lösungsarchiv.

Es wird also, wie oben erwähnt, immer nur die erste Lösung s_1 aus dem Lösungsarchiv gewählt. Danach wird die lineare Abweichung in der d -ten Dimension zu dieser Lösung mit der Gleichung (16) berechnet.

$$\delta_d = \xi \sum_{j=1}^k \frac{|s_{j_d} - s_{1_d}|}{k-1}, \quad j \in \{1, \dots, k\}, \quad d \in \{1, \dots, n\}, \quad (16)$$

Dabei ist n die Anzahl der Dimensionen des Problems.

Anschließend wird für jede Dimension wieder ein zufälliger Punkt aus dem Intervall

$$[s_d - \delta_d, s_d + \delta_d], \quad d \in \{1, \dots, n\}, \quad (17)$$

genommen. Damit haben wir auch hier, ausgehen von einem Lösungsarchiv, eine neue Lösung gewonnen.

Die Standardeinstellungen der Parameter sind, wie in Tabelle 2 zu sehen ist.

Tabelle 2: Parametereinstellungen ACO_ℝ-very-simple

Parameter	Symbol	Wert
Anzahl der Ameisen	m	1
Konvergenzgeschwindigkeit	ξ	0.95
Archivgröße	k	100

Da ACO_ℝ-simple den Lösungsfindungsprozess zwar deutlich vereinfacht, jedoch einen integralen Bestandteil, das Generieren von neuen Lösungen ausgehend von weniger guten Lösungen beibehält, also anstatt nur stur nach der besten Lösung zu gehen, auch in anderen Gebieten nach vielversprechenden Ergebnissen sucht, ist dieser Algorithmus, gemäß unserer Intuition, der Favorit zwischen den beiden Algorithmen zur Intervallauswertung.

4. Testfunktion

Es wurden, wie in Tabelle 4, Tabelle 5 und Tabelle 6 zu entnehmen, die dort aufgeführten Testfunktionen verwendet. Da in dieser Arbeit mit $ACO_{\mathbb{R}}$ verglichen wird, vergleichen sich $ACO_{\mathbb{R}}$ -simple und $ACO_{\mathbb{R}}$ -very-simple indirekt auch mit den anderen Verfahren, mit denen sich $ACO_{\mathbb{R}}$ verglichen hat.

Da ein direkter Vergleich über den Umfang dieser Arbeit aber hinaus gehen würde, wird hier nur mit $ACO_{\mathbb{R}}$ verglichen und bzgl. des Vergleiches mit den anderen Methoden auf *Socha und Dorigo* [6] verwiesen.

Die Testfunktionen sind alle aus [6] entnommen, um die beiden Algorithmen vergleichen zu können, bis auf Branin RCOS, Goldstein and Price, Griewangk, Hartmann und Shekel. Die Branin RCOS-Testfunktion wurde aus [10] entnommen, die Goldstein and Price-Testfunktion aus [11], Griewangk-Testfunktion wurde aus *Molga und Smutnicki* [5] entnommen, während die Shekel-Testfunktionen aus [7] und die Hartmann-Testfunktionen aus [8] und [9]. Sie stimmen mit denen aus [6] überein, bis auf einen Matrixeintrag bei der Hartmann(6,4)-Testfunktion, aber in den anderen Quellen waren die Optima angegeben und die Argumente der Optima, also ihre x-Werte, während in [6] nicht, weswegen die Entscheidung sich auf die anderen Quellen, bei den Angaben zu Optima und den dazugehörigen Argumenten zu verlassen, getroffen wurde.

Der Matrixeintrag ist $a_{15} = 1.7$, statt 1.5, da alle vorhandenen Quellen, bis auf [6], dies so angeben.

Ferner wurde festgestellt, dass die angegeben Optima in [7], [8] und [9] nicht mit den Optima übereinstimmen, wenn in die Funktion die Argumente der Optima, also ihre x-Werte, eingegeben werden. Deswegen werden die berechneten Optima genutzt, statt den angegeben, da die Algorithmen sonst keine Chance haben die Optima zu finden und für die aufgeführten Funktionen nicht konvergieren.

Die verwendeten Optima sind wie folgt aufgeführt.

Tabelle 3: berechnete Optima statt angegeben Optima

Funktion	Optima
Hartmann(3,4)	y= 7.421029683540756e-06 statt y= -3.86278
Hartmann(6,4)	y= 0.42294247223934295 statt y= -3.32237
Shekel(4,5)	y = -2.552128998986617 statt y = -10.1532
Shekel(4,7)	y = -2.561924906388908 statt y = -10.4029
Shekel(4,10)	y = -2.5727358977878936 statt y = -10.5364

Es konnte leider nicht festgestellt werden, ob die angegebenen Optima falsch sind oder ob die Berechnung und damit die Implementierung der Testfunktionen im Programm falsch ist. Wahrscheinlich ist die Implementierung fehlerhaft, aber nach gründlicher Kontrolle des Codes konnte ich leider keinen Fehler feststellen und nehme auch deswegen die berechneten Optima als Ausgangswerte für die Analyse der Güte der Algorithmen.

Positiv aufgefallen ist aber, dass bei der Suche nach den Fehlern die Algorithmen die berechneten Optima selbst dann fanden, wenn die Angegebenen als Ziel verwendet wurden. Die Verfahren brachen nur nicht ab, weil das Ziel eben eines war, was nicht erreicht werden konnte.

Ferner wurde das Abbruchkriterium für alle Testfunktionen, außer für die Maximierungsprobleme „Plane“ und „Diagonal Plane“, aus Tabelle 4 auf $|f - f^*| < \epsilon, \epsilon = 10^{-10}$ gesetzt, dabei ist f der momentane Funktionswert und f^* das bekannte Optimum. Für die Maximierungsprobleme wurde das Abbruchkriterium $f > \epsilon$ gesetzt, mit $\epsilon = 10^{10}$.

In *Socha und Dorigo* [6] wurde später das schwächere Abbruchkriterium $|f - f^*| < \epsilon_1 f + \epsilon_2, \epsilon_1 = \epsilon_2 = 10^{-4}$ verwendet, wobei ϵ_1 der relative Fehler und ϵ_2 der absolute Fehler ist. Um Vergleichswerte zu haben, wurde auch hier das schwächere Kriterium für Testfunktionen aus Tabelle 5 und 6 angewandt, wobei es etwas abgeändert wurde zu $|f - f^*| < |\epsilon_1 f + \epsilon_2|, \epsilon_1 = \epsilon_2 = 10^{-4}$. Die Änderung ist notwendig, da bei negativen Funktionswerten die linke Seite stets positiv bleibt, während die rechte negativ werden kann, sodass die Algorithmen nicht konvergieren.

Es ist noch zu bemerken, dass das Initialisierungsintervall, also das Intervall aus dem die ersten zufälligen k Lösungen entnommen werden, um das Lösungsarchiv zu füllen, von entscheidender Bedeutung ist. Wenn es unglücklich gewählt ist, liegen die Lösungen so um das Optimum, dass bei bestimmten Testproblemen die Algorithmusleistung stark zum positiven verzerrt wird. Ein so wichtiger Punkt wird hier nur deswegen nebensächlich erwähnt, da in *Socha und Dorigo* [6] darauf bereits eingegangen wurde, die Initialisierungsintervalle, wie sie dort aufgeführt sind, für gut befunden wurden und wir diese lediglich nur übernehmen müssen.

Wir bezeichnen die Initialisierungsintervalle hier jedoch als Definitionsbereiche, da die Funktionen initial über diesen definiert sind und führen sie nach dem Namen der Funktion an oder separat, wie in Tabelle 7.

Tabelle 4: Testfunktionen zum Vergleich mit Algorithmen, die probabilistisches Lernen einsetzen

Funktion, Definitionsbereich, Optimum	Formel
Cigar, $\vec{x} \in [-3, 7]^n, n = 10$, 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = x_1^2 + 10^4 \sum_{i=2}^n x_i^2$
Diagonal plane, $\vec{x} \in [0.5, 1.5]^n, n = 10$, 10^{10} bei mehreren	$f(x) = \frac{1}{n} \sum_{i=1}^n x_i$
Ellipsoid, $\vec{x} \in [-3, 7]^n, n = 10$, 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = \sum_{i=1}^n \left(100^{\frac{i-1}{n-1}} x_i\right)^2$
Paraboloid, $\vec{x} \in [-3, 7]^n, n = 10$, 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = \sum_{i=1}^n x^2$
Plane, $\vec{x} \in [0.5, 1.5]^n, n = 10$, 10^{10} bei $x_1 = 10^{10}$	$f(x) = x_1$
Rosenbrock, $\vec{x} \in [-3, 7]^n, n = 10$, 0 bei $\vec{x} = [1, 1, 1, \dots]$	$f(x) = \sum_{i=1}^{n-1} 100(x_i^2 - x_{i+1})^2 + (x_i - 1)^2$
Tablet, $\vec{x} \in [-3, 7]^n, n = 10$, 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = 10^4 x_1^2 + \sum_{i=2}^n x_i^2$

Tabelle 5: erster Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Probleme adaptiert wurden

Funktion, Definitionsbereich, Optimum	Formel
B_2 , $\vec{x} \in [-100, 100]^n, n = 2$, 0 bei $\vec{x} = [0, 0]$	$f(x) = x_1^2 + 2x_2^2 - \frac{3}{10}\cos(3\pi x_1) - \frac{2}{5}\cos(4\pi x_2) + 7/10$
Branin RCOS, $\vec{x} \in [-5, 15]^n, n = 2$, 0.397887 bei $\vec{x} = [-\pi, 12.275]$ od. $x = [\pi, 2.275]$ od. $x = [9.42478, 2.475]$	$f(x) = (x_2 - \frac{5}{4\pi}x_1^2 + \frac{5}{\pi}x_1 - 6)^2 + 10(1 - \frac{1}{8\pi})\cos(x_1) + 10$
Easom, $\vec{x} \in [-100, 100]^n, n = 2$, 0 bei $\vec{x} = [\pi \cdot k + \pi/2]$, $\pi \cdot k + \pi/2], k \in \mathbb{Z}$	$f(x) = -\cos(x_1) \cos(x_2) e^{(x_1 - \pi)^2 (x_2 - \pi)^2}$
De Jong, $\vec{x} \in [-5.12, 5.12]^n, n = 3$, 0 bei $\vec{x} = [0, 0, 0]$	$f(x) = x_1^2 + x_2^2 + x_3^2$
Goldstein, $\vec{x} \in [-2, 2]^n$ and Price, $n = 2$, 3 bei $\vec{x} = [0, 1]$	$f(x) = (1 + (x_1 + x_2 + 1)^2(19 - 14x_1 + 13x_1^2 - 14x_2 + 6x_1x_2 + x_2^2))(30 + (2x_1 - 3x_2)^2 \cdot (18 - 32x_1 + 12x_1^2 - 48x_2 - 36x_1x_2 - 27x_2^2))$
Griewangk, $\vec{x} \in [-600, 600]^n$, $n = 10$; 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = \sum_{i=1}^n \frac{x_i^2}{4000} - \prod_{i=1}^n \cos(\frac{x_i}{\sqrt{i}}) - 1$
Martin and Gaddy, $\vec{x} \in [-20, 20]^n, n = 2$ 0 bei $\vec{x} = [5, 5]$	$f(x) = (x_1 - x_2)^2 + \left(\frac{x_1 + x_2 - 10}{3}\right)^2$
Paraboloid(6), $\vec{x} \in [-5.12, 5.12]^n$, $n=6$; 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = \sum_{i=1}^n x_i^2$
Rosenbrock(2,5), $\vec{x} \in [-5, 5]^n$, $n = 2, 5$; 0 bei $\vec{x} = [1, 1, 1, \dots]$	$f(x) = \sum_{i=1}^{n-1} 100(x_i^2 - x_{i+1})^2 + (x_i - 1)^2$
Zakharov, $\vec{x} \in [-5, 10]^n$, $n = 2, 5$; 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = \sum_{j=1}^n x_j^2 + \left(\sum_{j=1}^n \frac{jx_j^2}{2}\right)^2 + \left(\sum_{j=1}^n \frac{jx_j^2}{2}\right)^4$

Bei Funktionen, welche in mehreren Dimensionen getestet wurden, stehen die getesteten Dimensionen in Klammern hinter der Funktion. Dasselbe gilt für Funktionen, die sich in der getesteten Dimension von der Dimension aus Tabelle 4 unterscheiden.

Tabelle 6: zweiter Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Probleme adaptiert wurden

Funktion	Formel
Hartmann(3,4)	$f_{H_{3,4}}(x) = - \sum_{i=1}^4 c_i e^{-\sum_{j=1}^3 a_{ij}(x_j - p_{ij})^2}$ $a_{ij} = \begin{pmatrix} 3.0 & 10.0 & 30.0 \\ 0.1 & 10.0 & 35.0 \\ 3.0 & 10.0 & 30.0 \\ 0.1 & 10.0 & 35.0 \end{pmatrix} \quad c_i = \begin{pmatrix} 1.0 \\ 1.2 \\ 3.0 \\ 3.2 \end{pmatrix}$ $p = \begin{pmatrix} 0.3689 & 0.1170 & 0.2673 \\ 0.4699 & 0.4387 & 0.7470 \\ 0.1091 & 0.8732 & 0.5547 \\ 0.0381 & 0.5743 & 0.8828 \end{pmatrix}$
Hartmann(6,4)	$f_{H_{3,4}}(x) = - \sum_{i=1}^4 c_i e^{-\sum_{j=1}^6 a_{ij}(x_j - p_{ij})^2}$ $a_{ij} = \begin{pmatrix} 10.0 & 3.0 & 17.0 & 3.5 & 1.7 & 8.0 \\ 0.05 & 10.0 & 17.0 & 0.1 & 8.0 & 14.0 \\ 3.0 & 3.5 & 1.7 & 10.0 & 17.0 & 8.0 \\ 17.0 & 8.0 & 0.05 & 10.0 & 0.1 & 14.0 \end{pmatrix} \quad c_i = \begin{pmatrix} 1.0 \\ 1.2 \\ 3.0 \\ 3.2 \end{pmatrix}$ $p = \begin{pmatrix} 0.1312 & 0.1696 & 0.5569 & 0.0124 & 0.8283 & 0.5886 \\ 0.2329 & 0.4135 & 0.8307 & 0.3736 & 0.1004 & 0.9991 \\ 0.2348 & 0.1451 & 0.3522 & 0.2883 & 0.3047 & 0.6650 \\ 0.4047 & 0.8828 & 0.8732 & 0.5743 & 0.1091 & 0.0381 \end{pmatrix}$
Shekel(4,k; k = 5,7,10)	$f(x) = - \sum_{i=1}^k (\sum_{j=1}^4 (x_j - A_{ji})^2 + c_i)^{-1}$ $A = \begin{pmatrix} 4.0 & 4.0 & 4.0 & 4.0 \\ 1.0 & 1.0 & 1.0 & 1.0 \\ 8.0 & 8.0 & 8.0 & 8.0 \\ 6.0 & 6.0 & 6.0 & 6.0 \\ 3.0 & 7.0 & 3.0 & 7.0 \\ 2.0 & 9.0 & 2.0 & 9.0 \\ 5.0 & 5.0 & 3.0 & 3.0 \\ 8.0 & 1.0 & 8.0 & 1.0 \\ 6.0 & 2.0 & 6.0 & 2.0 \\ 7.0 & 3.6 & 7.0 & 3.6 \end{pmatrix} \quad c_i = \begin{pmatrix} 0.1 \\ 0.2 \\ 0.2 \\ 0.4 \\ 0.4 \\ 0.6 \\ 0.3 \\ 0.7 \\ 0.5 \\ 0.5 \end{pmatrix}$

Tabelle 7: zweiter Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Probleme adaptiert wurden

Funktion, Definitionsbereich	Optimum
Hartmann(3,4), $\vec{x} \in [0, 1]^n, n = 3$	7.421029683540756e-06 bei $\vec{x} = [0.114614, 0.555649, 0.852547]$
Hartmann(6,4), $\vec{x} \in [0, 1]^n, n = 6$	0.42294247223934295 bei $\vec{x} = [0.20169, 0.150011, 0.476874,$ 0.275332, 0.311652, 0.6573]
Shekel(4,5) $\vec{x}, \in [0, 1]^n, n = 4$	-2.552128998986617 bei $\vec{x} = [4, 4, 4, 4]$
Shekel(4,7) $\vec{x}, \in [0, 1]^n, n = 4$	-2.561924906388908 bei $\vec{x} = [4, 4, 4, 4]$
Shekel(4,10) $\vec{x}, \in [0, 1]^n, n = 4$	2.5727358977878936 bei $\vec{x} = [4, 4, 4, 4]$

5. Einfluss der Parameter

Bevor wir $ACO_{\mathbb{R}}$, $ACO_{\mathbb{R}}$ -simple und $ACO_{\mathbb{R}}$ -very-simple mit einander vergleichen, betrachten wir den Einfluss der Parameter in $ACO_{\mathbb{R}}$ -simple und $ACO_{\mathbb{R}}$ -very-simple. Die dieser Arbeit beiliegenden Algorithmen sind von zwei bzw. drei Parametern beeinflusst. Im Fall $ACO_{\mathbb{R}}$ -simple sind es q , k und ξ und im Fall von $ACO_{\mathbb{R}}$ -very-simple sind es k und ξ , wie respektive in den Gleichungen (12) und (14) bzw. der Gleichung (16) zu sehen ist – zumindest nach Konstruktion. Tests haben jedoch gezeigt, dass es vier bzw. drei Parameter sind, nämlich, dass die Anzahl der Ameisen auch eine wesentliche Rolle spielt.

Der Einfluss der Parameter wird außerdem geprüft, da es sich bei $ACO_{\mathbb{R}}$ -simple und $ACO_{\mathbb{R}}$ -very-simple lediglich um Intervallauswertungsmethoden handelt und hier der Einfluss der Parameter anders sein kann, als bei $ACO_{\mathbb{R}}$, wir also zuerst überprüfen, ob er gleich ist oder nicht. Es besteht nämlich die Möglichkeit, dass aufgrund der zum Teil radikalen Einfachheit, vor allem von $ACO_{\mathbb{R}}$ -very-simple, gegenüber $ACO_{\mathbb{R}}$, der Einfluss der Parameter, im Vergleich zu $ACO_{\mathbb{R}}$, einen anderen Charakter annimmt.

Es wird jedoch davon ausgegangen, da sich alle Methoden doch ähneln, dass der Einfluss vergleichbar oder sogar der gleiche ist.

Abschließend ist noch festzustellen, dass durch die Betrachtung der Einflüsse der Parameter auf $ACO_{\mathbb{R}}$ -simple und $ACO_{\mathbb{R}}$ -very-simple, eine Art Parameteroptimierung bzw. Parameter Tuning durchgeführt wird. Das ist zwar nicht das Ziel der Betrachtung, aber wenn wir die Einflüsse verschiedener Parameterwerte auf die Leistungsfähigkeit der Algorithmen untersuchen, untersuchen wir auch gleichzeitig die Güte der Wahl der Para-

meterwerte. Es wird jedoch an dieser Stelle nochmal betont, dass das Ziel der Betrachtung, der Einflüsse der Parameter, die Untersuchung der Ähnlichkeit der Varianten zur Lösungserzeugung ist.

5.1. Einfluss der Anzahl der Ameisen

Normalerweise wird bei einem Ameisenalgorithmus die Anzahl der Ameisen klein gehalten, aber sie erreicht niemals eins. $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple arbeiten jedoch, wie in der Abbildung 1, Abbildung 6, Abbildung 7 und Abbildung 8 zu sehen, am besten mit nur einer einzigen Ameise, also wenn $m=1$.

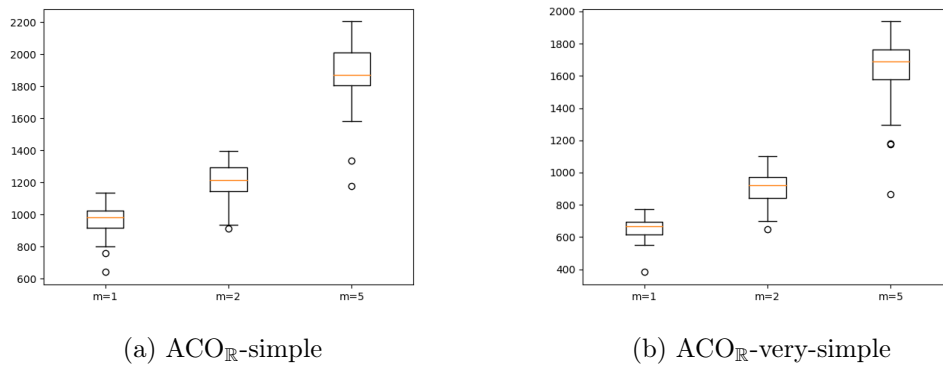


Abbildung 1: Ellipsoid Vergleich m

Da in $\text{ACO}_{\mathbb{R}}$, m kein Parameter war und dort somit nicht diskutiert wurde, können wir leider keine Aussage darüber treffen, ob die Algorithmen $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple in ihrem Verhalten bzgl. des Parameters m ähnlich zu $\text{ACO}_{\mathbb{R}}$ sind.

5.2. Einfluss des Parameters q

Wie in $\text{ACO}_{\mathbb{R}}$, erwarten wir in $\text{ACO}_{\mathbb{R}}$ -simple, dass q die Lokalität der Suche steuert - je kleiner q , desto mehr werden die besseren Lösungen zum Generieren neuer Lösungen bevorzugt, desto schneller konvergiert der Algorithmus. Dies ist auch der Fall, was anhand von Abbildung 2, Abbildung 9, Abbildung 10 und Abbildung 11 zu sehen ist.

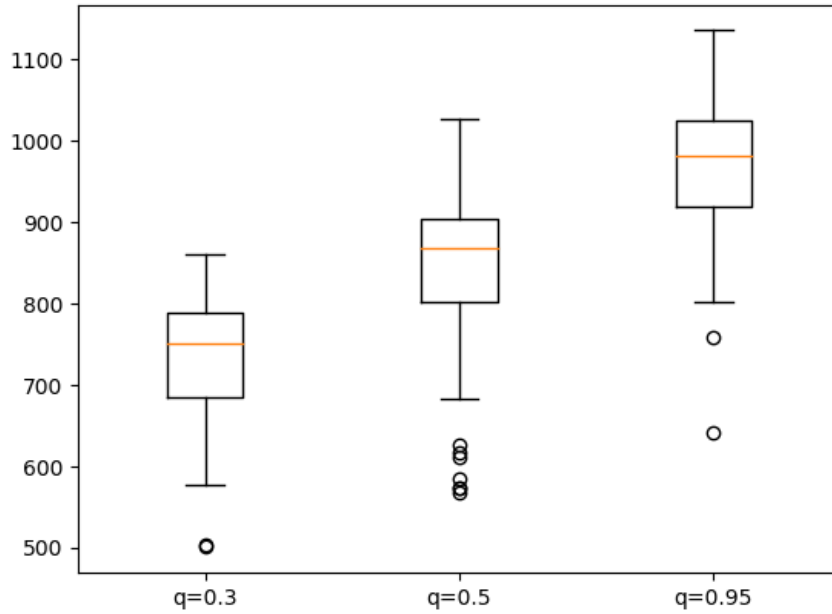


Abbildung 2: Ellipsoid Vergleich q

Der Algorithmus ist also in seinem Verhalten bzgl. des Parameters q ähnlich zu $\text{ACO}_{\mathbb{R}}$.

Für $\text{ACO}_{\mathbb{R}}$ -very-simple gibt es keine Abbildungen bzgl. des Einflusses des Parameters q , da dieser Parameter bei $\text{ACO}_{\mathbb{R}}$ -very-simple entfällt. Jedoch stellen wir fest, dass der Algorithmus sehr schnell konvergiert, viel schneller als $\text{ACO}_{\mathbb{R}}$ -simple, wie in Tabelle 10 zu sehen. Ferner können wir die Abwesenheit des Parameters q mit einem hinreichend kleinen q gleichstellen, da, wenn q sehr klein ist, nur die erste, also beste Lösung als Ausgangspunkt für die folgende Lösungserzeugung dient und $\text{ACO}_{\mathbb{R}}$ -simple selbst viel schneller konvergiert, als beim Standardwert $q=0.95$. Wir interpretieren also die schnelle Konvergenz von $\text{ACO}_{\mathbb{R}}$ -very-simple als den Einfluss eines hinreichend kleinen q .

Beide Algorithmen, $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple, sind also in ihrem Verhalten bzgl. des Parameters q ähnlich zu $\text{ACO}_{\mathbb{R}}$.

5.3. Einfluss des Parameters k

Wie zu erwarten war, funktionieren beide Algorithmen, $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple, mit größeren k besser. Es wurde für alle späteren Durchläufe $k=100$ gesetzt.

Da in $\text{ACO}_{\mathbb{R}}$, k kein Parameter war und dort somit nicht diskutiert wurde, können wir leider keine Aussage darüber treffen, ob die Algorithmen $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple in ihrem Verhalten bzgl. des Parameters k ähnlich zu $\text{ACO}_{\mathbb{R}}$ sind.

5.4. Einfluss des Parameters ξ

In $\text{ACO}_{\mathbb{R}}$ hat der Parameter ξ eine ähnliche Wirkung wie die Verdunstungsrate des Pheromons. Je größer ξ , desto langsamer konvergiert der Algorithmus. In $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple vergrößert ξ das Suchintervall direkt, wodurch es einen noch stärkeren Einfluss auf die Konvergenz des Algorithmus haben sollte, jedoch stellt sich heraus, dass ξ nur einen geringen Einfluss, bei $\text{ACO}_{\mathbb{R}}$ -simple, auf die Testfunktion „Ellipsoid“ hat, wie Abbildung 3 zeigt.

Für die Testfunktionen „Cigar“, „Paraboloid“ und „Tablet“ wurde das Ziel jedoch nur bei $\xi = 0.95$ und $\xi = 1.5$ erreicht. Was wiederum einen sehr starken Einfluss nahelegt.

Diese Beobachtung lässt sich dadurch erklären, dass „Ellipsoid“ die Testfunktion ist, für welche $\text{ACO}_{\mathbb{R}}$ -simple mit großem Abstand die besten Werte liefert, die Leistung des Algorithmus also für diese Testfunktion vom ausgewählten Parameter wenig beeinflusst wird. Somit betrachten wir, wie der Algorithmus sich bei anderen Testfunktionen verhält.

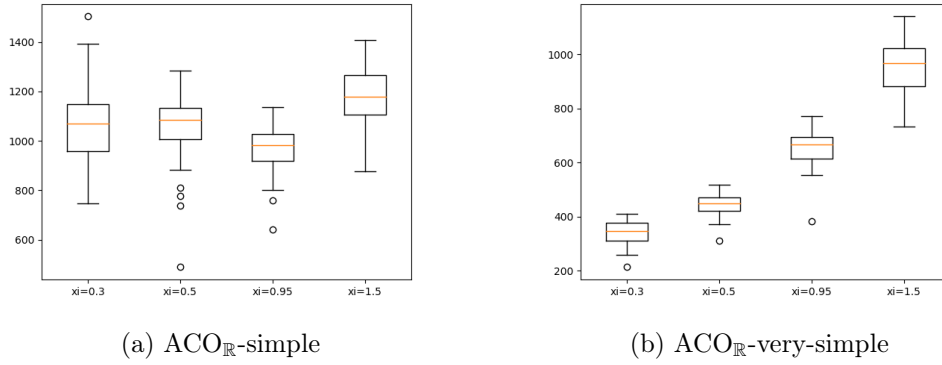


Abbildung 3: Ellipsoid Vergleich ξ

Wenn wir also das Verhalten von $\text{ACO}_{\mathbb{R}}$ -simple bei der Testfunktion „Ellipsoid“ als Eigenheit des Algorithmus ausklammern und uns auf die restlichen Ergebnisse konzentrieren, stellen wir fest, dass beide Algorithmen, sowohl $\text{ACO}_{\mathbb{R}}$ -simple als auch $\text{ACO}_{\mathbb{R}}$ -very-simple, bei kleinerem ξ nicht konvergieren und bei größerem ξ langsamer konvergieren, wie Abbildung 12, Abbildung 13 und Abbildung 14 zeigen. Beide Algorithmen sind also in ihrem Verhalten bzgl. des Parameters ξ ähnlich zu $\text{ACO}_{\mathbb{R}}$.

5.5. Auswertung der Einflüsse der Parameter

Wir stellen fest, dass die Einflüsse der Parameter q und ξ in $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple ähnlich sind, wie die Einflüsse dieser Parameter in $\text{ACO}_{\mathbb{R}}$, wie wir es erwartet haben.

Die Einflüsse sind jedoch nur gleich für den Parameter q . Für den Parameter ξ sind sie verschieden, wie Abbildungen 3, 12, 13 und 14 zeigen. Denn wie wir den Abbildungen entnehmen können, konvergieren weder $\text{ACO}_{\mathbb{R}}$ -simple noch $\text{ACO}_{\mathbb{R}}$ -very-simple für kleine ξ , während wir davon ausgehen, dass dies bei $\text{ACO}_{\mathbb{R}}$ nicht der Fall ist, da in *Socha und Dorigo* [6] erwähnt wurde, dass $\text{ACO}_{\mathbb{R}}$ für größere ξ langsamer konvergiert. Wir gehen also davon aus, dass $\text{ACO}_{\mathbb{R}}$ für kleinere ξ nur schneller konvergiert und dies einfach nicht genauer untersucht wurde.

Zum Schluss stellen wir noch fest, dass der Parameter ξ erstaunlicherweise optimal gewählt wurde, der Parameter q sollte allerdings möglichst klein sein. Es war zwar nicht das Ziel der Betrachtung eine Parameteroptimierung durchzuführen, jedoch ist dies ein Nebenprodukt, dass bei einer Diskussion der Parameter unweigerlich entsteht.

Da wir jedoch ausdrücklich die Algorithmen $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple in ihrer original konzipierten Fassung untersuchen wollen und zudem der einzige Unterschied zwischen den beiden die Größe des Parameters q ist, bei $\text{ACO}_{\mathbb{R}}$ -very-simple können wir sagen, q ist sehr klein, müssen wir das Nebenprodukt der Untersuchung der Parametereinflüsse ignorieren. Denn wenn wir es mit einarbeiten, stellen wir fest, dass ein kleineres q besser ist, wir $\text{ACO}_{\mathbb{R}}$ -very-simple also zweimal untersuchen und hier gerade die Unterschiede zwischen $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple von Interesse sind.

6. Vergleich zweier Intervallberechnungen für $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple

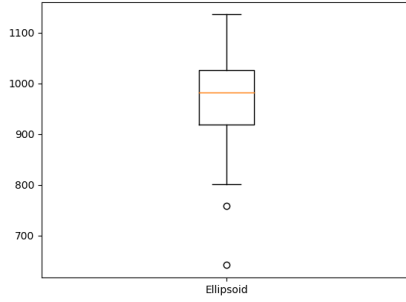
Nun gilt es, den Begriff der Distanzberechnung zu untersuchen. Da es Unstimmigkeiten bzgl. der Benennung der Intervallberechnung gab, werden diese im folgenden geklärt. In *Socha und Dorigo* [6] wurde

$$\delta_d = \xi \sum_{j=1}^k \frac{|s_j^d - s_i^d|}{k-1}, \quad j \in \{1, \dots, k\}, \quad d \in \{1, \dots, n\}, \quad (18)$$

als Standardabweichung bezeichnet. Dies ist jedoch als lineare Abweichung bekannt. Wir untersuchen also im Folgenden, ob es bessere Ergebnisse liefert, statt der linearen Abweichung, Gleichung (18), die Standardabweichung, Gleichung (19) zu nehmen.

$$\delta_d = \xi \sqrt{\sum_{j=1}^k \frac{|s_j^d - s_i^d|}{k-1}}, \quad j \in \{1, \dots, k\}, \quad d \in \{1, \dots, n\}, \quad (19)$$

Es stellt sich heraus, dass die Intervalle, welche die lineare Abweichung liefert, also die Berechnung anhand Gleichung (18), die besseren sind, wie Abbildung 4 und Abbildung 5 verdeutlichen.

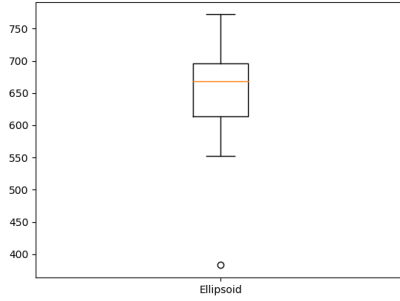


(a) $\text{ACO}_{\mathbb{R}}\text{-simple}$

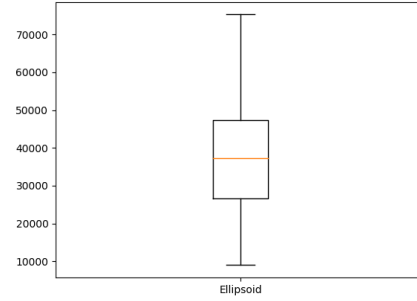


(b) $\text{ACO}_{\mathbb{R}}\text{-simple}$

Abbildung 4: Vergleich zwischen linearer und Standardabweichung (Ellipsoid)



(a) $\text{ACO}_{\mathbb{R}}\text{-very-simple}$



(b) $\text{ACO}_{\mathbb{R}}\text{-very-simple}$

Abbildung 5: Vergleich zwischen linearer und Standardabweichung (Ellipsoid)

Dabei zeigt der linke Boxplot (a) immer die Anzahl der notwendigen Funktionsauswertungen, mit linearer Abweichung, um das bekannte Ergebnis zu erreichen und der rechte Boxplot (b), die mit Standardabweichung.

Abbildungen 4 und 5 sind auch leider die einzigen Abbildungen, welche als Vergleichsmöglichkeit für lineare und Standardabweichung für Funktionen aus Tabelle 4 herangezogen werden können, da beide Algorithmen, $\text{ACO}_{\mathbb{R}}\text{-simple}$ und $\text{ACO}_{\mathbb{R}}\text{-very-simple}$, für Testfunktionen aus Tabelle 4, außer für „Ellipsoid“, mit der Standardabweichung als Intervallauswertung nicht konvergierte.

Andere Funktionen wurden nach diesen Ergebnisses nicht mehr getestet, da die Funktionen aus Tabelle 4 die einfachen Funktionen aus den verfügbaren Testproblemen darstellen und wenn für diese Funktionen keine Konvergenz zu beobachten ist, dann wird für die anderen erst recht keine zu beobachten sein.

Somit kann geschlussfolgert werden, dass die Intervallberechnung anhand Formel (18), also der linearen Abweichung, die bessere ist.

7. Vergleich von $ACO_{\mathbb{R}}$, $ACO_{\mathbb{R}}$ -simple und $ACO_{\mathbb{R}}$ -very-simple

Wie in der Betrachtung der Einflüsse der Parameter auf $ACO_{\mathbb{R}}$ -simple und $ACO_{\mathbb{R}}$ -very-simple sich bereits abgezeichnet hat, gehen wir davon aus, dass $ACO_{\mathbb{R}}$ -very-simple bei einfachen Testproblemen der bessere Algorithmus ist, da wenn eine Funktion hinreichend einfach ist, es nicht notwendig ist, bereits gefundene gute Lösungen zu einer noch besseren Lösung aufwendig zu verbinden und kann man zu einfacheren Mitteln der Lösungsfindung greifen. Deswegen gehen wir davon aus, dass $ACO_{\mathbb{R}}$ -very-simple im Vergleich in den Tabellen 8 und 9 als der bessere Algorithmus hervorgehen wird.

In den Tabellen dieses Abschnittes werden ferner als Bestwert immer die Bestwerte aller Algorithmen aus *Socha und Dorigo* [6] betrachtet und zusätzlich der Algorithmen $ACO_{\mathbb{R}}$ -simple und $ACO_{\mathbb{R}}$ -very-simple, wie sie in den Tabellen aufgeführt werden, weswegen in Tabelle 8 und Tabelle 9 verschiedene Bestwerte zu sehen sind, da in Tabelle 8 $ACO_{\mathbb{R}}$ -very-simple nicht gelistet ist und somit seine Bestwerte in den Vergleich nicht einfließen.

Tabelle 8: Vergleich der Ergebnisse zwischen $ACO_{\mathbb{R}}$ -simple und $ACO_{\mathbb{R}}$ basierend auf *Socha und Dorigo* [6] und damit auch auf *Kern et al., 2004* [3]

Testfunktion	$ACO_{\mathbb{R}}$ -simple	$ACO_{\mathbb{R}}$	Bestwert
Cigar	3.5 (13516)	1.4 (5376)	1.0 (3840)
Diagonal Plane	50.2 (8531)	1.0 (170)	1.0 (170)
Ellipsoid	1.0 (947)	12.2 (11570)	1.0 (947)
Paraboloid	7.3 (10041)	1.1 (1570)	1.0 (1370)
Plane	63.9 (11176)	1.0 (175)	1.0 (175)
Rosenbrock	∞	*1.1 (7909)	1.0 (7190)
Tablet	4.2 (10812)	1.0 (2567)	1.0 (2567)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo ein * steht, wurde die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht, und dort wo ∞ steht wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Wie in Tabelle 8 zu sehen, ist $ACO_{\mathbb{R}}$ der bessere Algorithmus. $ACO_{\mathbb{R}}$ -simple mag zwar bei einer Testfunktion vorne liegen, jedoch ist $ACO_{\mathbb{R}}$ -simple zu schlecht bei den Maximierungsproblemen und auch nicht so gut bei den anderen Testfunktionen, vor allem bei „Rosenbrock“, wo der Algorithmus das Optimum gar nicht findet.

Tabelle 9: Vergleich der Ergebnisse zwischen $\text{ACO}_{\mathbb{R}}$ -very-simple und $\text{ACO}_{\mathbb{R}}$ basierend auf *Socha und Dorigo* [6] und damit auf *Kern et al., 2004* [3]

Testfunktion	$\text{ACO}_{\mathbb{R}}$ -very-simple	$\text{ACO}_{\mathbb{R}}$	Bestwert
Cigar	1.0 (2037)	2.6 (5376)	1.0 (2037)
Diagonal Plane	1.0 (148)	1.1 (170)	1.0 (148)
Ellipsoid	1.0 (644)	18 (11570)	1.0 (644)
Paraboloid	1.1 (1483)	1.1 (1570)	1.0 (1370)
Plane	1.0 (131)	1.3 (175)	1.0 (131)
Rosenbrock	∞	*1.1 (7909)	1.0 (7190)
Tablet	1.0 (1751)	1.5 (2567)	1.0 (1751)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo ein * steht, wurde die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht und dort wo ∞ steht wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Wie in Tabelle 9 zu sehen, ist $\text{ACO}_{\mathbb{R}}$ -very-simple, bei einfachen Testproblemen, der bessere Algorithmus in allen Testproblemen, bis auf „Rosenbrock“. Hätte $\text{ACO}_{\mathbb{R}}$ -very-simple auch bei diesem Problem eine Lösung gefunden, wäre er, für einfache Testprobleme, der klare Favorit gewesen. So aber kann man sagen, dass die Algorithmen, bei einfachen Testproblemen, ebenbürtig sind, besonders, da $\text{ACO}_{\mathbb{R}}$ bei den anderen Testfunktionen nur wenig schlechter ist, außer „Ellipsoid“.

Auch wenn aus den Tabellen 8 und 9 bereits entnommen werden kann, dass von den beiden Algorithmen $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple, $\text{ACO}_{\mathbb{R}}$ -very-simple der bessere ist, betrachten wir in Tabelle 10 nochmal eine direkte Gegenüberstellung, um ein noch klareres Ergebnis zu erhalten.

Tabelle 10: Vergleich der Ergebnisse zwischen $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple

Testfunktion	$\text{ACO}_{\mathbb{R}}$ -simple	$\text{ACO}_{\mathbb{R}}$ -very-simple
Cigar	6.6 (13516)	1.0 (2037)
Diagonal Plane	57.6 (8531)	1.0 (148)
Ellipsoid	1.5 (947)	1.0 (644)
Paraboloid	6.8 (10041)	1.0 (1483)
Plane	85.3 (11176)	1.0 (131)
Rosenbrock	∞	∞
Tablet	6.2 (10812)	1.0 (1751)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Dort, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Wie in Tabelle 10 nochmal deutlich zu sehen ist, ist $\text{ACO}_{\mathbb{R}}$ -very-simple, bei einfach Testproblemen, der bessere Algorithmus.

Tabelle 11: Zweiter Teil des Vergleiches der Ergebnisse zwischen $ACO_{\mathbb{R}}$ -simple und $ACO_{\mathbb{R}}$ basierend auf *Socha und Dorigo* [6] und damit auf *Kern et al., 2004* [3]

Testfunktion	$ACO_{\mathbb{R}}$ -simple	$ACO_{\mathbb{R}}$	Bestwert
B_2	5.3 (2272)	1.3 (544)	1.0 (430)
Branin RCOS	6.5 (1600)	3.5 (858)	1.0 (245)
Easom	-	1.0 [98%] (772)	1.0 (772)
De Jong	2.9 (965)	1.0 ($\sim$ 392)	1.0 (329)
Goldstein and Price	4.0 (917)	1.7 (384)	1.0 (231)
Griewangk	∞	1.0 [61%] (1390)	1.0 (1390)
Hartmann(3,4)	-	1.0 (342)	1.0 (342)
Hartmann(6,4)	2.3 (1714)	1.0 (722)	1.0 (722)
Martin and Gaddy	2.8 (974)	1.0 (345)	1.0 (345)
Paraboloid(6)	3.0 (2308)	1.6 (781)	1.0 (781)
Rosenbrock(2)	15.2 (7317)	1.7 (820)	1.0 (480)
Rosenbrock(5)	∞	1.2 [97%] (2487)	1.0 (2142)
Shekel(4,5)	2.7 (1642)	1.3 [57%] (787)	1.0 [76%] (610)
Shekel(4,7)	2.3 (1595)	1.1 [79%] (748)	1.0 [83%] (680)
Shekel(4,10)	2.4 (1559)	1.1 [81%] (715)	1.0 [83%] (650)
Zakharov(2)	3.4 (664)	1.7 (332)	1.0 (195)
Zakharov(5)	5.7 (4157)	1.0 (727)	1.0 (727)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Bei der Funktion „De Jong“ konnte für $ACO_{\mathbb{R}}$ die genaue Anzahl an Funktionsauswertungen nicht ermittelt werden.

Wie in Tabelle 11 zu sehen, ist $ACO_{\mathbb{R}}$ -simple der schlechtere Algorithmus, er ist sogar bei einer Funktion viel schlechter als $ACO_{\mathbb{R}}$.

Die Easom-Testfunktion konnte leider nicht angegeben werden, da bei dem Initialisierungsintervall zu große Werte für den Algorithmus auftreten und es zu einem Überlauf kommt. Bei kleineren Initialisierungsintervallen konvergiert der Algorithmus jedoch schnell.

Die Hartmann(3,4)-Testfunktion konnte leider nicht angegeben werden, da die Anzahl der durchschnittlichen Funktionsauswertungen bei dieser Testfunktion bei 4 lag und es ist eher wahrscheinlich, dass bei dem Auswerten, Berechnen oder Initialisieren ein Fehler aufgetreten ist, als dass der Algorithmus wirklich nur 4 Funktionsauswertungen braucht.

Auf der anderen Seite, wenn die Genauigkeit auf $|f - f^*| < |\epsilon_1 f + \epsilon_2|$, $\epsilon_1 = \epsilon_2 = 10^{-10}$ angehoben wird, braucht der Algorithmus mehrere Tausend Funktionsauswertungen (5000-12000) und findet das Optimum sogar manchmal nicht, was viel eher dafür spricht, dass es hauptsächlich an der Initialisierung gepaart mit der geringen Anforderung an die Genauigkeit liegt und es kein Fehler der Berechnung ist.

Weiter haben Tests ergeben, dass selbst bei einer Archivgröße von $k=10$, statt $k=100$, das selbe Ergebnis beobachtet werden kann, was die Initialisierung ausschließt, da 10 zufällige Lösungen, selbst wenn es gute sind, den Effekt nicht erklären können und vor allem nicht bei allen 50 Durchläufen, da auch mal schlechte Lösungen ins Archiv hätten aufgenommen werden müssen. Das Ergebnis konnte jedoch konstant beobachtet werden, unabhängig von der Archivgröße.

Damit lässt sich die geringe Anforderung an die Genauigkeit als Quelle des Effektes feststellen. Auf der anderen Seite ist das Ergebnis einfach zu gut, um lediglich mit einer geringen Anforderung an die Genauigkeit erklärt werden zu können. Ich gehe davon aus, dass es noch andere Faktoren gibt und führe das Ergebnis in der Tabelle 11 nicht auf.

Tabelle 12: Zweiter Teil des Vergleiches der Ergebnisse zwischen $ACO_{\mathbb{R}}$ -very-simple und $ACO_{\mathbb{R}}$ basierend auf *Socha und Dorigo* [6] und damit auf *Kern et al., 2004* [3]

Testfunktion	$ACO_{\mathbb{R}}$ -very-simple	$ACO_{\mathbb{R}}$	Bestwert
B_2	1.7 (747)	1.3 (544)	1.0 (430)
Branin RCOS	3.0 (747)	3.5 (858)	1.0 (245)
Easom	-	1.0 [98%] (772)	1.0 (772)
De Jong	1.0 (319)	1.2 ($\sim$ 392)	1.0 (319)
Goldstein and Price	1.5 (344)	1.7 (384)	1.0 (231)
Griewangk	12.3 [44%] (17062)	1.0 [61%] (1390)	1.0 (1390)
Hartmann(3,4)	-	1.0 (342)	1.0 (342)
Hartmann(6,4)	1.0 (377)	1.9 (722)	1.0 (377)
Martin and Gaddy	1.0 (356)	1.0 (345)	1.0 (345)
Paraboloid(6)	1.0 (490)	1.6 (781)	1.0 (490)
Rosenbrock(2)	2.5 (1195)	1.7 (820)	1.0 (480)
Rosenbrock(5)	18.4 [54%] (39345)	1.2 [97%] (2487)	1.0 (2142)
Shekel(4,5)	1.0 (445)	1.8 [57%] (787)	1.0 (445)
Shekel(4,7)	1.0 (444)	1.8 [79%] (748)	1.0 (444)
Shekel(4,10)	1.0 (443)	1.6 [81%] (715)	1.0 (443)
Zakharov(2)	1.5 (293)	1.7 (332)	1.0 (195)
Zakharov(5)	1.0 (694)	1.7 (727)	1.0 (694)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Bei der Funktion „De Jong“ konnte für $ACO_{\mathbb{R}}$ die genaue Anzahl an Funktionsauswertungen nicht ermittelt werden.

Wie in Tabelle 12 zu sehen, ist es nicht mehr klar, welcher der Algorithmen vorzuziehen ist. $ACO_{\mathbb{R}}$ ist bei vier Testfunktionen der beste Algorithmus, während $ACO_{\mathbb{R}}$ -very-simple bei Sieben. Beide Algorithmen sind jedoch bei einem Teil der Testfunktionen schlechter als das Optimum.

Die Easom-Testfunktion konnte leider nicht angegeben werden, da bei den Initialisierungsintervall zu große Werte für den Algorithmus auftreten und es zu einem Überlauf kommt. Bei kleineren Initialisierungsintervallen konvergiert der Algorithmus jedoch sehr schnell.

Die Hartmann(3,4) Testfunktion konnte leider nicht angegeben werden, da die Anzahl der durchschnittlichen Funktionsauswertungen bei dieser Testfunktion bei 2 lag und es

ist eher wahrscheinlich, dass bei dem Auswerten, Berechnen oder Instanzieren ein Fehler aufgetreten ist, als dass wirklich nur 2 Funktionsauswertungen nötig waren. Der Fehler konnte leider nicht gefunden werden.

Auf der anderen Seite, wenn die Genauigkeit auf $|f - f^*| < |\epsilon_1 f + \epsilon_2|$, $\epsilon_1 = \epsilon_2 = 10^{-10}$ angehoben wird, braucht der Algorithmus auch hier mehrere Tausend Funktionsauswertungen (1000-5000), was viel eher dafür spricht, dass es hier genauso hauptsächlich an der Initialisierung gepaart mit der geringen Anforderung an die Genauigkeit liegt und es kein Fehler der Berechnung ist.

Hier ist die Argumentation die selbe, nämlich dass es an der geringen Genauigkeit liegt, dass so ein Ergebnis beobachtet werden kann.

Allerdings entscheide ich mich genauso wie in Tabelle 11 das Ergebnis in der Tabelle 12 nicht aufzuführen.

Wie bereits erwähnt, geht es über den Umfang dieser Arbeit hinaus, $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ mit anderen Algorithmen aus *Socha und Dorigo* [6] als $\text{ACO}_{\mathbb{R}}$ zu vergleichen, weswegen nur betrachtet wird, ob einer der beiden Algorithmen der beste ist oder nicht, wie in Tabelle 9 und Tabelle 12 verdeutlicht.

Deswegen lässt es sich, durch diese eingeschränkte Sichtweise, leicht urteilen, dass $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ der Favorit ist. Er mag $\text{ACO}_{\mathbb{R}}$ zwar bei einer einfacheren Testfunktion (Rosenbrock) unterlegen sein, wie in Tabelle 9 zu sehen, ist jedoch auch im Schnitt besser, wenn es um schwerere Testfunktionen geht, wie in Tabelle 12 klar wird. $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ mag bei zwei der schwereren Testfunktionen (Griewangk und Rosenbrock(5)) wesentlich mehr Funktionsauswertungen brauchen als $\text{ACO}_{\mathbb{R}}$, ist jedoch bei den besonders schweren Testfunktionen, den Hartmann- und Shekelfunktionen, $\text{ACO}_{\mathbb{R}}$ überlegen.

Wie auch bei den einfachen Testfunktionen, um das Ergebnis klarer zu sehen, vergleichen wir $\text{ACO}_{\mathbb{R}}\text{-simple}$ und $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ direkt in Tabelle 13

Tabelle 13: Zweiter Teil des Vergleiches der Ergebnisse zwischen $ACO_{\mathbb{R}}$ -simple und $ACO_{\mathbb{R}}$ -very-simple

Testfunktion	$ACO_{\mathbb{R}}$ -simple	$ACO_{\mathbb{R}}$ -very-simple
B_2	3.0 (2272)	1.0 (747)
Branin RCOS	2.1 (1600)	1.0 (747)
Easom	-	-
De Jong	3.0 (965)	1.0 (319)
Goldstein and Price	2.7 (917)	1.0 (344)
Griewangk	∞	1.0 [44%] (17062)
Hartmann(3,4)	-	-
Hartmann(6,4)	5.0 (1714)	1.0 (377)
Martin and Gaddy	2.6 (974)	1.0 (356)
Paraboloid(6)	4.7 (2308)	1.0 (490)
Rosenbrock(2)	6.2 (7317)	1.0 (1195)
Rosenbrock(5)	∞	1.0 [54%] (39345)
Shekel(4,5)	3.7 (1642)	1.0 (445)
Shekel(4,7)	3.6 (1595)	1.0 (444)
Shekel(4,10)	3.5 (1559)	1.0 (443)
Zakharov(2)	2.3 (664)	1.0 (293)
Zakharov(5)	6.0 (4157)	1.0 (694)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Wie in Tabelle 13 nochmal deutlich zu sehen, ist $ACO_{\mathbb{R}}$ -very-simple, auch bei schwereren Testproblemen, der bessere Algorithmus.

8. Konvergenzbetrachtung

Zwar ist Tabelle 12 bereits sehr aussagekräftig, aber bei einem Urteil, dass sich ausschließlich auf diese Tabelle stützt, geht verloren, dass die Leistung, die $ACO_{\mathbb{R}}$ -very-simple liefert, schon ausreichend sein kann, selbst wenn der Algorithmus nicht konvergiert, gemessen daran, wie einfach, somit schnell der Algorithmus ist, im Vergleich zu $ACO_{\mathbb{R}}$. Es liegen zwar keine Daten zur Zeitkomplexität von $ACO_{\mathbb{R}}$ vor, aber wir können davon ausgehen, dass $ACO_{\mathbb{R}}$ -very-simple schneller ist.

Deswegen betrachten wir in diesem Abschnitt, ob $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ gegen Werte konvergiert, die den bekannten Optimalwerten der Testprobleme nahe sind.

Es mag zwar stimmen, dass bei Algorithmen zur Lösung stetiger Probleme, es mehr Sinn macht, die Anzahl der Funktionsauswertungen als Maßstab der Güte eines Verfahrens zu nehmen, jedoch ist der in *Socha und Dorigo* [6] beschriebene Algorithmus selbst von keiner kleinen Zeitkomplexität. Weswegen das dort gebrachte Argument, die Laufzeit zum Vergleich nicht hinzuziehen, an Stärke verliert.

Es ist immer noch sehr stark, da alle dafür angebrachte Argumente wahr und valide sind, wie dass die Laufzeit von der verwendeten Programmiersprache abhängt, vom Compiler oder von dem Rechner und somit man zwar die Laufzeiten messen könnte aber diese weniger Aussagekräftig ist als die Anzahl der Funktionsauswertungen aber die Verfasser der Quelle hätten die Laufzeit trotzdem mit angeben können, da es bei dem Vergleich mit den dieser Arbeit beiliegenden Verfahren sehr interessant wäre zu sehen, wie viel schneller die hier vorgestellten Algorithmen sind und ob ihre Geschwindigkeit ein Argument für ihre Verwendung sein könnte.

Deswegen wäre an dieser Stelle die Zeitkomplexität von $\text{ACO}_{\mathbb{R}}$ besonders interessant gewesen, da, wenn der Algorithmus $\text{ACO}_{\mathbb{R}}$ doch noch langsamer ist als ohnehin erwartet, die Wahl zwischen den Algorithmen, noch mehr in Richtung $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ ausfallen würde. Betrachten wir nämlich den Fall, wo viele Arbeitsschritte mit geringerer Präzision nacheinander schnell abgearbeitet werden müssen und $\text{ACO}_{\mathbb{R}}$ noch langsamer als erwartet ist, wäre $\text{ACO}_{\mathbb{R}}\text{-very-simple}$, $\text{ACO}_{\mathbb{R}}$ tatsächlich eindeutig vorzuziehen, da in den folgenden Abschnitten klar zu sehen sein wird, dass $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ dem geforderten Optimalwert des Testproblems, wo das Verfahren keine Lösung findet, hinreichend nahe kommt.

Somit werden wir am Ende der Konvergenzbetrachtung festhalten können, $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ ist, bei geringerer Anforderung an die Genauigkeit der Lösung, $\text{ACO}_{\mathbb{R}}$ noch mehr als sonst vorzuziehen, sofern $\text{ACO}_{\mathbb{R}}$ langsam genug ist.

8.1. Rosenbrock

Die Rosenbrockfunktion nimmt ihren Optimalwert $y=0$ an bei $\vec{x} = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]$.

Der durchschnittlich berechnete Funktionswert bei 10000 Funktionsauswertungen war ungefähr $y=13$, mit $x_i \in [-1, 1] \forall i \in \{1, 2, 3, \dots, 10\}$

Wie zu sehen, ist der Funktionswert etwas weiter entfernt von dem Optimum aber $\vec{x}$ ist nah dran am Argument des Optimums.

Somit können wir schlussfolgern, dass $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ hinreichend gut ist, auch bei der Funktion, wo der Algorithmus keine Lösung findet.

9. Kurzzusammenfassung

Es ist erstaunlich, wie die Vereinfachung abschneidet. Zwar mag $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ nicht immer der beste Algorithmus sein, aber er ist der beste bei 12 von 24 Testfunktion, also bei der Hälfte aller Testfunktion. Damit ist er $\text{ACO}_{\mathbb{R}}$ überlegen. Natürlich sind fünf dieser Testfunktionen einfach, aber selbst bei den schweren Funktionen weiß $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ zu überzeugen, insbesondere, da er sehr schnell ist und unter bestimmten Umständen sogar auf ein exaktes Ergebnis verzichten kann.

Insbesondere ist es bemerkenswert, dass es der Algorithmus $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ ist, der sich mit $\text{ACO}_{\mathbb{R}}$ messen kann und nicht $\text{ACO}_{\mathbb{R}}\text{-simple}$, obwohl unsere Intuition hier gerade das gegenteilige Ergebnis vorhergesagt hat.

Abschließend lässt sich sagen, dass durch $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ ein weiterer ausgezeichneter Algorithmus der Klasse der Ameisenalgorithmen beigetreten ist, dazu noch einer, der so einfach ist, wie es nur geht und damit die Frage aus der Einleitung beantwortet. Es geht viel einfacher als bisher – eben very simple.

Ferner ist das dieser Arbeit beiliegende Programm so geschrieben, dass es sich leicht um weitere Suchverfahren und Testfunktionen erweitern lässt.

Es ist so geschrieben, dass nur das Verfahren selbst programmiert und der Rest so beibehalten werden kann – das heißt die Testfunktionen können für das neue Verfahren weiter verwendet werden, ohne weitere Änderung am Code, genauso wie die Funktionen zum generieren der Boxplots. Ebenso einfach können neuen Testfunktionen eingebunden werden. Es ist lediglich notwendig die Testfunktion hinzuzufügen.

Das macht das Ausprobieren neuer Suchalgorithmen bedeutend einfacher und schneller. Zur Anschauung, wie einfach, wurde in das dieser Arbeit beiliegende Programm der Standard-PSO Algorithmus eingebaut.

Literatur

- [1] BILCHEV, G. ; PARMEE, I. C.: The ant colony metaphor for searching continuous design spaces. In: *AISB workshop on evolutionary computing* Springer, 1995, S. 25–39
- [2] GUNTSCH, M. ; MIDDENDORF, M. : A population based approach for ACO. In: *Workshops on Applications of Evolutionary Computation* Springer, 2002, S. 72–81
- [3] KERN, S. ; MÜLLER, S. D. ; HANSEN, N. ; BÜCHE, D. ; OCENASEK, J. ; KOU-MOUTSAKOS, P. : Learning probability distributions in continuous evolutionary algorithms—a comparative review. In: *Natural Computing* 3 (2004), Nr. 1, S. 77–112
- [4] MIDDENDORF, M. : Verfahren der Schwarmintelligenz. In: *Vorlesung Verfahren der Schwarmintelligenz* (2017)
- [5] MOLGA, M. ; SMUTNICKI, C. : Test functions for optimization needs. In: *Test functions for optimization needs* 101 (2005)
- [6] SOCHA, K. ; DORIGO, M. : Ant colony optimization for continuous domains. In: *European journal of operational research* 185 (2008), Nr. 3, S. 1155–1173
- [7] SURJANOVIC, S. ; BINGHAM, D. : Virtual Library of Simulation Experiments: Test Function and Datasets. In: <https://www.sfu.ca/~ssurjano/shekel.html> (abgerufen am 17.08.2019)
- [8] SURJANOVIC, S. ; BINGHAM, D. : Virtual Library of Simulation Experiments: Test Function and Datasets. In: <https://www.sfu.ca/~ssurjano/hart3.html> (abgerufen am 17.08.2019)
- [9] SURJANOVIC, S. ; BINGHAM, D. : Virtual Library of Simulation Experiments: Test Function and Datasets. In: <https://www.sfu.ca/~ssurjano/hart6.html> (abgerufen am 17.08.2019)
- [10] SURJANOVIC, S. ; BINGHAM, D. : Virtual Library of Simulation Experiments: Test Function and Datasets. In: <https://www.sfu.ca/~ssurjano/branin.html> (abgerufen am 19.08.2019)
- [11] SURJANOVIC, S. ; BINGHAM, D. : Virtual Library of Simulation Experiments: Test Function and Datasets. In: <https://www.sfu.ca/~ssurjano/goldpr.html> (abgerufen am 19.08.2019)

Tabellenverzeichnis

1.	Parametereinstellungen ACO _R -simple	8
2.	Parametereinstellungen ACO _R -very-simple	9
3.	berechnete Optima statt angegeben Optima	10
4.	Testfunktionen zum Vergleich mit Algorithmen, die probabilistisches Lernen einsetzen	12
5.	erster Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenal- gorithmen verwandten Methoden und Metaheuristiken, die für stetige Probleme adaptiert wurden	13
6.	zweiter Teil der Testfunktionen zum Vergleich mit anderen mit Amei- senalgorithmen verwandten Methoden und Metaheuristiken, die für ste- tige Probleme adaptiert wurden	14
7.	zweiter Teil der Testfunktionen zum Vergleich mit anderen mit Amei- senalgorithmen verwandten Methoden und Metaheuristiken, die für ste- tige Probleme adaptiert wurden	15
8.	Vergleich der Ergebnisse zwischen ACO _R -simple und ACO _R basierend auf <i>Socha und Dorigo</i> [6] und damit auch auf <i>Kern et al., 2004</i> [3]	21
9.	Vergleich der Ergebnisse zwischen ACO _R -very-simple und ACO _R basie- rend auf <i>Socha und Dorigo</i> [6] und damit auf <i>Kern et al., 2004</i> [3]	22
10.	Vergleich der Ergebnisse zwischen ACO _R -simple und ACO _R -very-simple	23
11.	Zweiter Teil des Vergleiches der Ergebnisse zwischen ACO _R -simple und ACO _R basierend auf <i>Socha und Dorigo</i> [6] und damit auf <i>Kern et al., 2004</i> [3]	24
12.	Zweiter Teil des Vergleiches der Ergebnisse zwischen ACO _R -very-simple und ACO _R basierend auf <i>Socha und Dorigo</i> [6] und damit auf <i>Kern et al., 2004</i> [3]	26
13.	Zweiter Teil des Vergleiches der Ergebnisse zwischen ACO _R -simple und ACO _R -very-simple	28

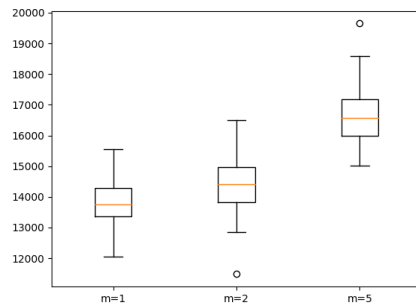
Abbildungsverzeichnis

1.	Ellipsoid Vergleich m	16
2.	Ellipsoid Vergleich q	17
3.	Ellipsoid Vergleich ξ	18
4.	Vergleich zwischen linearer und Standardabweichung (Ellipsoid)	20
5.	Vergleich zwischen linearer und Standardabweichung (Ellipsoid)	20
6.	Cigar Vergleich m	
7.	Paraboloid Vergleich m	
8.	Tablet Vergleich m	
9.	Cigar Vergleich q	
10.	Paraboloid Vergleich q	
11.	Tablet Vergleich q	

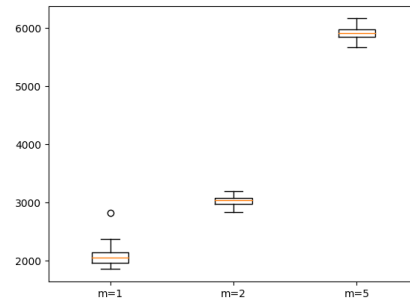
12. Cigar Vergleich xi
13. Paraboloid Vergleich xi
14. Tablet Vergleich xi

Anlagen

A. Einfluss der Anzahl der Ameisen

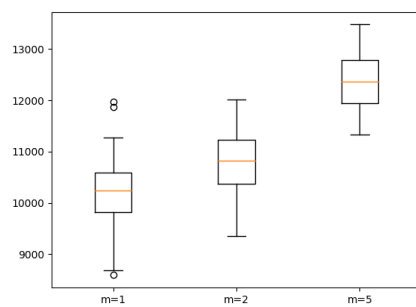


(a) ACO_R-simple

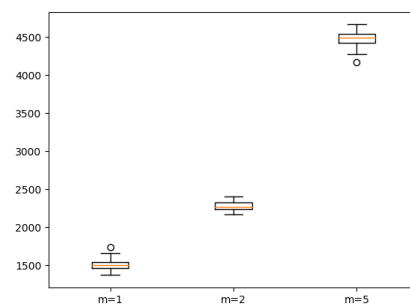


(b) ACO_R-very-simple

Abbildung 6: Cigar Vergleich m

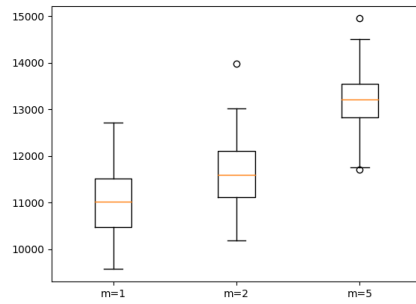


(a) ACO_R-simple

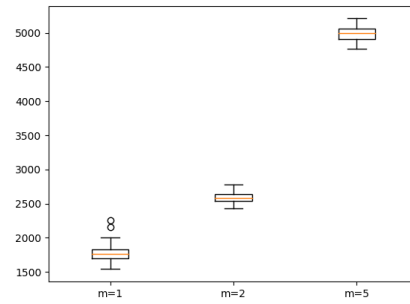


(b) ACO_R-very-simple

Abbildung 7: Paraboloid Vergleich m



(a) $\text{ACO}_{\mathbb{R}}\text{-simple}$



(b) $\text{ACO}_{\mathbb{R}}\text{-very-simple}$

Abbildung 8: Tablet Vergleich m

B. Einfluss des Parameters q

Da bei $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ der Parameter q entfällt, beziehen sich alle Abbildungen in diesem Abschnitt nur auf $\text{ACO}_{\mathbb{R}}\text{-simple}$.

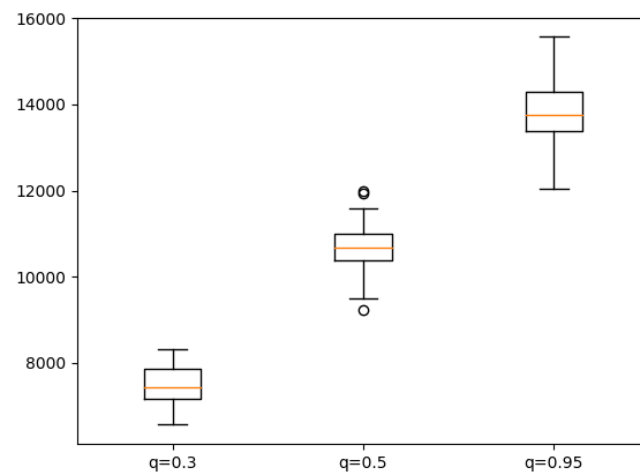


Abbildung 9: Cigar Vergleich q

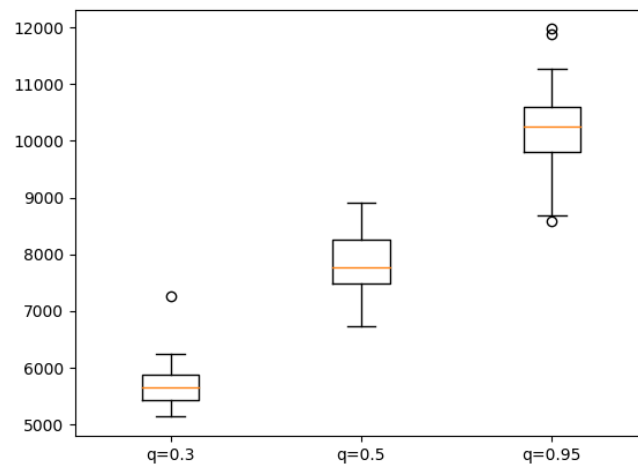


Abbildung 10: Paraboloid Vergleich q

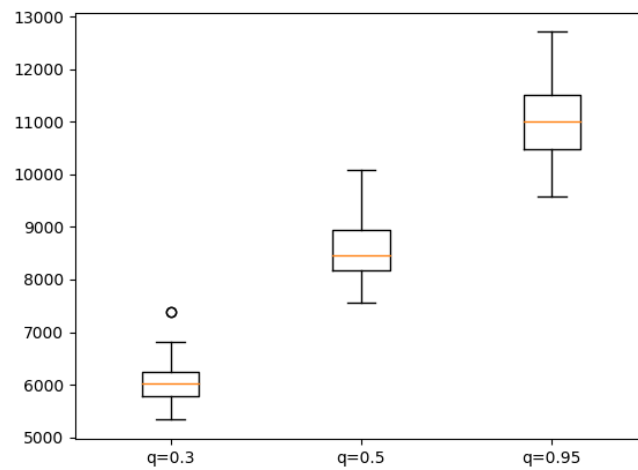
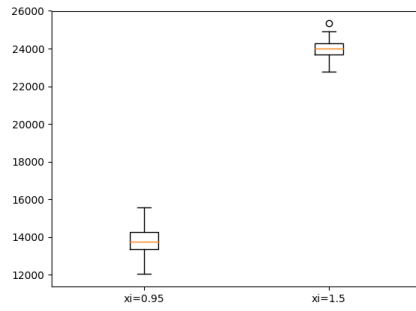


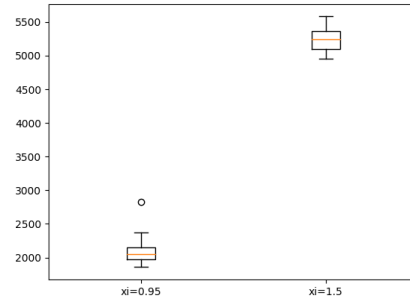
Abbildung 11: Tablet Vergleich q

C. Einfluss des Parameters ξ

Die in diesem Abschnitt enthaltenen Abbildungen haben immer nur zwei Box Plots, da beide Algorithmen für die beiden anderen Testwerte $\xi=0.3$ und $\xi=0.5$ nicht konvergierten.

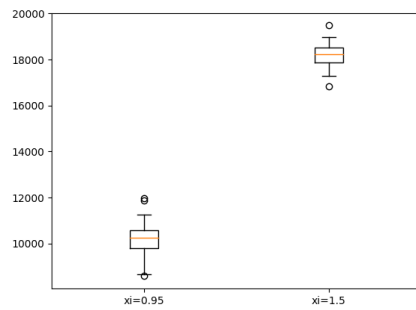


(a) $\text{ACO}_{\mathbb{R}}$ -simple

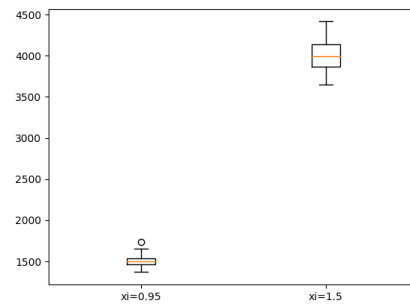


(b) $\text{ACO}_{\mathbb{R}}$ -very-simple

Abbildung 12: Cigar Vergleich xi

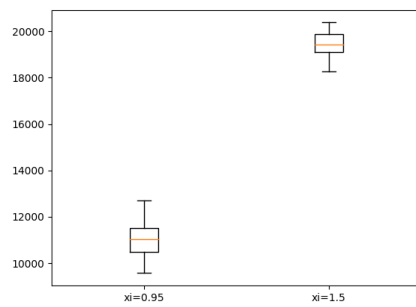


(a) $\text{ACO}_{\mathbb{R}}$ -simple

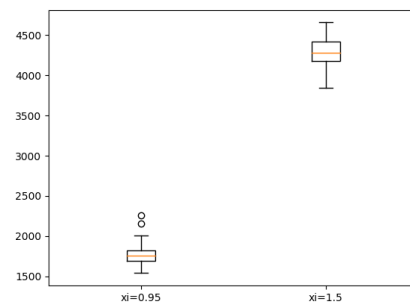


(b) $\text{ACO}_{\mathbb{R}}$ -very-simple

Abbildung 13: Paraboloid Vergleich xi



(a) $\text{ACO}_{\mathbb{R}}$ -simple



(b) $\text{ACO}_{\mathbb{R}}$ -very-simple

Abbildung 14: Tablet Vergleich xi

Erklärung

Ich versichere, dass ich die vorliegende Arbeit selbstständig und nur unter Verwendung der angegebenen Quellen und Hilfsmittel angefertigt habe, insbesondere sind wörtliche oder sinngemäße Zitate als solche gekennzeichnet. Mir ist bekannt, dass Zuwiderhandlung auch nachträglich zur Aberkennung des Abschlusses führen kann. Ich versichere, dass das elektronische Exemplar mit den gedruckten Exemplaren übereinstimmt.

Ort: Leipzig

Datum:

Unterschrift: